Contents

1 General **1**
- 1.1 Compilation . . . 1
- 1.2 Template . . . 1

2 Miscellaneous **2**
- 2.1 Custom Sort . . . 2
- 2.2 Bounds . . . 2

3 Number Theory **2**
- 3.1 Binary Exponentiation . . . 2

4 Bits **2**
- 4.1 Gray Code . . . 2

5 Graphs **2**
- 5.1 Dijkstra . . . 2
- 5.2 Binary Lifting . . . 3
- 5.3 LCA . . . 3

6 Data Structures **3**
- 6.1 Segment Tree . . . 3
- 6.2 Lazy Segment Tree . . . 4
- 6.3 DSU . . . 4
- 6.4 Sparse Table . . . 5

7 Math **5**
- 7.1 Linear Sieve . . . 5
- 7.2 FFT . . . 5
- 7.3 NTT . . . 6
- 7.4 Multiply . . . 6

8 Strings **7**
- 8.1 KMP . . . 7
- 8.2 Manacher . . . 7

9 Geometry **8**
- 9.1 Picks Theorem (Code) . . . 8

10 Fórmulas **8**
- 10.1 Fórmulas de sumatorias . . . 8
- 10.2 Picks Theorem . . . 8

1 General

1.1 Compilation

```
g++ problem.cpp -std=c++23 -O2 -Wall -Wextra -Wshadow -Wconversion -Wfloat-equal -Wduplicated-cond -Wlogical-op -o
    r && ./r < in.txt > out.txt
```

1.2 Template

```
#include<bits/stdc++.h>
#define debug(x) cerr<<#x<<":"<<x<<endl;
typedef long long ll;
using namespace std;
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    return 0;
}
```

2 Miscellaneous

2.1 Custom Sort

```
void sol(){
    vector<pair<int,int>> v;
    sort(v.begin(),v.end(),[](pair<int,int> a,pair<int,int> b){
        if(a.first==b.first) return a.second<b.second;
        return a.first>b.first;
    });
}
```

2.2 Bounds

```
int lowerBound(vector<int> &v,int target){
  int l=0;
  int r=v.size();
  while(l<r){
    int mid=l+(r-l)/2;
    /*To change to upper use <= instead of <*/
    if(v[mid]<target)
        l=mid+1;
    else
        r=mid;
  }
  return l;
}
```

3 Number Theory

3.1 Binary Exponentiation

```
ll binpow(ll a, ll b){
    ll res=1;
    while(b){
        if(b&1) res*=a;
        a*=a;
        b>>=1;
    }
    return res;
}
```

4 Bits

4.1 Gray Code

```
int g (int n) {
    return n ^ (n >> 1);
}
int rev_g (int g) {
  int n = 0;
  for (; g; g >>= 1)
    n ^= g;
  return n;
}
```

5 Graphs

5.1 Dijkstra

```
typedef pair<ll,ll> Esp;
void dijkstra(){
    vector<vector<Esp>> graph(n+1);
    vector<ll> dist(n+1,INF);
    priority_queue<Esp,vector<Esp>,greater<Esp>> pq;
    pq.push({0,1});
    dist[1]=0;
    while(!pq.empty()){
        auto [n_dist,node]=pq.top(); pq.pop();
        if(n_dist>dist[node]) continue;
        for(auto [v2,cost]:graph[node]){
            if(dist[v2]>n_dist+cost){
                dist[v2]=dist[node]+cost;
                pq.push({dist[v2],v2});
            }
```

```
        }
    }
}
```

5.2 Binary Lifting

```
void sol(){
    int n; cin>>n;
    int l=__lg(n)+2;
    vector<vector<int>> up(l,vector<int>(n+1));
    //up[x][y] desde y hasta (1<<x)
    for(int i=2;i<=n;i++){
        int x; cin>>x;
        up[1][i]=x;
    }
    for(int i=2;i<l;i++){
        for(int j=1;j<=n;j++){
            int ant=up[i-1][up[i-1][j]];
            up[i][j]=ant;
        }
    }
    function<int(int,int)> go_up=[&](int x, int k){
        int current=x;
        int aux=1;
        while(k){
            if(k&1) current=up[aux][current];
            aux++;
            k>>=1;
        }
        return current;
    };
}
```

5.3 LCA

```
void sol(){
    int n,q; cin>>n>>q;
    vector<vector<int>> graph(n+1);
    int l=__lg(n)+2;
    vector<vector<int>> up(l,vector<int>(n+1));
    //Calculate binary lifting
    vector<int> dp(n+1);
    //Calculate the dept from root
    while(q--){
        int a,b; cin>>a>>b;
        if(dp[a]<dp[b])swap(a,b);
        int dif=dp[a]-dp[b];
        a=go_up(a,dif);
        if(b==a){cout<<a<<"\n"; continue;}
        //b is father of a
        for(int i=l-1;i>=1;i--){
            int aux_a=up[i][a];
            int aux_b=up[i][b];
            if(aux_a!=aux_b){
                a=aux_a;
                b=aux_b;
            }
        }
        cout<<up[1][a]<<"\n";
    }
}
```

6 Data Structures

6.1 Segment Tree

```
struct Node{
    Node operator+(const Node &other) const{
        //merge
    }
};
struct SegmentTree {
    int n;
    vector<Node> st;
    SegmentTree(int n, vector<ll>& v) {
        this->n = n;
        st.assign(2 * n, Node());
        for (int i = 0; i < n; i++) st[n + i] = Node(v[i]);
        for (int i = n - 1; i > 0; i--)
            st[i] = st[i << 1]+st[i << 1 | 1];
    }
    void update(int pos, ll value) {
        pos += n;
```

```
        st[pos] = value;
        for (int i = pos >> 1; i > 0; i >>= 1) {
            st[i] = st[i << 1]+st[i << 1 | 1];
        }
    }
    Node query(int l, int r) {
        Node resL = Node(), resR = Node();
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1){
                resL = resL+ st[l]; l++;
            }
            if (r & 1){
                r--; resR =st[r]+ resR;
            }
        }
        //[l,r)
        return resL+ resR;
    }
};
```

6.2 Lazy Segment Tree

```
struct SegLazy{
    int n; vector<Node> st; vector<ll>lazy;
    SegLazy(int n, vector<ll> &v){
        this->n=n;
        st.assign(4*n,Node());
        lazy.assign(4*n,0);
        build(1,0,n-1,v);
    }
    void build(int node, int l, int r, const vector<ll>&v){
        if(l==r){
            st[node]=Node(v[l]);
            return;
        }
        int mid=(l+r)/2;
        build(node*2,l,mid,v);
        build(node*2+1,mid+1,r,v);
        st[node]=st[node*2]+st[node*2+1];
    }
    void apply(int node, int l,int r, ll val){
        st[node].val+=val*(r-l+1);
        lazy[node]+=val;
    }
    void push(int node, int l, int r){
        if(lazy[node]!=0 && l!=r){
            int mid=(l+r)/2;
            apply(node*2,l,mid,lazy[node]);
            apply(node*2+1,mid+1,r,lazy[node]);
            lazy[node]=0;
        }
    }
    void update(int node, int l, int r, int ql, int qr, ll val){
        if(ql>r || qr<l) return;
        if(ql<=l && r<=qr){
            apply(node,l,r,val);
            return;
        }
        push(node,l,r);
        int mid=(l+r)/2;
        update(node*2,l,mid,ql,qr,val);
        update(node*2+1,mid+1,r,ql,qr,val);
        st[node]=st[node*2]+st[node*2+1];
    }
    Node query(int node, int l, int r, int ql, int qr){
        if(ql>r || qr<l) return Node();
        if(ql<=l && r<=qr) return st[node];
        push(node,l,r);
        int mid=(l+r)/2;
        return query(node*2,l,mid,ql,qr)+query(node*2+1,mid+1,r,ql,qr);
    }
    void update(int l, int r, ll val){
        update(1,0,n-1,l,r,val);
    }
    Node query(int l, int r){
        return query(1,0,n-1,l,r);
    }//all in [l,r]
};
```

6.3 DSU

```
struct DSU {
    vector<int> parent, sz;
    DSU(int n) {
        parent.resize(n);
        sz.assign(n,1);
        for (int i=0;i<n;i++)
            parent[i]=i;
```

```
    }
    int find_set(int v) {
        if(v==parent[v]) return v;
        return parent[v]=find_set(parent[v]);
    }
    bool join(int a, int b) {
        a = find_set(a);
        b = find_set(b);
        if(a!=b){
            if(sz[a]<sz[b]) swap(a, b);
            parent[b]=a;
            sz[a]+=sz[b];
            return true;
        }
        return false;
    }
    int size(int a){
        return sz[find_set(a)];
    }
};
```

6.4 Sparse Table

```
struct SparseTable{
    int n;
    vector<int> log2;
    vector<vector<int>>st;
    SparseTable(const vector<int>&a){
        n=a.size();
        log2.resize(n+1);
        log2[1]=0;
        for(int i=2;i<=n;i++) log2[i]=log2[i/2]+1;
        int k=log2[n]+1;
        st.assign(n,vector<int>(k));
        for(int i=0;i<n;i++) st[i][0]=a[i];
        for(int j=1;j<k;j++){
            for(int i=0;i+(1<<j)<=n;i++){
                st[i][j]=max(
                    st[i][j-1],
                    st[i+(1<<(j-1))][j-1]
                );
            }
        }
    }
    int query(int l, int r){
        int j=log2[r-l+1];
        return max(st[l][j],st[r-(1<<j)+1][j]);
    }
};
```

7 Math

7.1 Linear Sieve

```
vector<int> mindiv;
vector<int> primes;
void sieve(int N) {
    mindiv = vector<int>(N+1, 0);
    primes = vector<int>();
    for(int i=2; i <= N; ++i) {
        if(mindiv[i] == 0) {
            mindiv[i] = i;
            primes.push_back(i);
        }
        for(int j=0; j < primes.size() and primes[j] <= mindiv[i] and primes[j]*i <= N; ++j) {
            mindiv[i*primes[j]] = primes[j];
        }
    }
}
```

7.2 FFT

```
using cd = complex<double>;
const double PI = acos(-1);

int reverse(int num, int lg_n){
    int res = 0;
    for (int i = 0; i < lg_n; i++){
        if (num & (1 << i))
            res |= 1<<(lg_n-1-i);
    }
    return res;
}
```

```
12
13  void fft(vector<cd> & a, bool invert) {
14      int n = a.size();
15      int lg_n = 0;
16      while ((1 << lg_n) < n) lg_n++;
17
18      for (int i = 0; i < n; i++) {
19          if (i < reverse(i, lg_n))
20              swap(a[i], a[reverse(i, lg_n)]);
21      }
22      for (int len = 2; len <= n; len <<= 1) {
23          double ang = 2 * PI / len * (invert ? -1 : 1);
24          cd wlen(cos(ang), sin(ang));
25          for (int i = 0; i < n; i += len) {
26              cd w(1);
27              for (int j = 0; j < len / 2; j++) {
28                  cd u = a[i+j], v = a[i+j+len/2] * w;
29                  a[i+j] = u + v;
30                  a[i+j+len/2] = u - v;
31                  w *= wlen;
32              }
33          }
34      }
35      if (invert) {
36          for (cd & x : a) x/=n;
37      }
38  }
```

7.3 NTT

```
1   const int mod = 7340033;
2   const int root = 5;
3   const int root_1 = 4404020;
4   const int root_pw = 1 << 20;
5   /*
6       const int mod = 998244353;
7       const int root = 31;
8       const int root_1 = 128805723;
9       const int root_pw = 1 << 23;
10
11      const int mod = 1004535809;
12      const int root = 12289;
13      const int root_1 = 313564925;
14      const int root_pw = 1 << 21;
15
16      const int mod = 469762049;
17      const int root = 2187;
18      const int root_1 = 410692747;
19      const int root_pw = 1 << 26;
20  */
21  void fft(vector<int> & a, bool invert) {
22      int n = a.size();
23      for (int i = 1, j = 0; i < n; i++) {
24          int bit = n >> 1;
25          for (; j & bit; bit>>=1) j ^= bit;
26          j ^= bit;
27
28          if(i<j) swap(a[i], a[j]);
29      }
30
31      for (int len = 2; len <= n; len <<= 1) {
32          int wlen = invert ? root_1 : root;
33          for (int i = len; i < root_pw; i <<= 1)
34              wlen = (int)(1LL * wlen * wlen % mod);
35
36          for (int i = 0; i < n; i += len) {
37              int w = 1;
38              for (int j = 0; j < len / 2; j++) {
39                  int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w % mod);
40                  a[i+j] = u + v < mod ? u + v : u + v - mod;
41                  a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
42                  w = (int)(1LL * w * wlen % mod);
43              }
44          }
45      }
46      if (invert) {
47          int n_1 = inverse(n, mod);
48          for (int & x : a)
49              x = (int)(1LL * x * n_1 % mod);
50      }
51  }
```

7.4 Multiply

```
1   vector<int> multiply(vector<int> const& a, vector<int> const& b) {
2       vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
3       int n = 1;
4       while (n < a.size() + b.size())
```

```
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);
    // For nnt just return fa
    // and apply MOD to fa[i]
    // If you want return result
    // Clean the right zeros
    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());

    return result;
}
```

8 Strings

8.1 KMP

```
vector<int> genLps(string &cad){
    int len=0;
    int j=1;
    int n=cad.size();
    vector<int> lps(n);
    lps[0]=0;
    while(j<n){
        if(cad[len]==cad[j]){
            len++;
            lps[j]=len;
            j++;
        }
        else{
            if(len!=0) len=lps[len-1];
            else{
                lps[j]=0;
                j++;
            }
        }
    }
    return lps;
}
vector<int> kmp(string &text, string &pat){
    vector<int> lps=genLps(pat);
    vector<int> ans;
    int i=0;
    int j=0;
    int n=text.size();
    while(i<n){
        if(pat[j]==text[i]){
            i++; j++;
            if(j==pat.size()){
                ans.push_back(i-j);
                j=lps[j-1];
            }
        }
        else{
            if(j!=0) j=lps[j-1];
            else i++;
        }
    }
    return ans;
}
```

8.2 Manacher

```
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for(int i = 1; i <= n; i++) {
        if(i <= r) {
            p[i] = min(r-i, p[l + (r - i)]);
        }
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l=i-p[i], r=i+p[i];
        }
    }
```

```
    return vector<int>(begin(p) + 1, end(p) - 1);
}
```

9 Geometry

9.1 Picks Theorem (Code)

```
struct Point{
    ll x,y;
    ll cross(const Point& o) const {return x*o.y-y*o.x;}
};
pair<ll,pair<ll,ll>> picks_theorem(vector<Point>&p){
    ll double_area=0, B=0;
    int n=p.size();
    for(int i=0,j=n-1;i<n;j=i++){
        double_area+=p[j].cross(p[i]);
        ll dx=abs(p[i].x-p[j].x);
        ll dy=abs(p[i].y-p[j].y);
        B+=gcd(dx,dy);
    }
    double_area=abs(double_area);
    ll I=(double_area-B+2)/2;
    //Real area = double_area/2.0
    return {double_area,{I,B}};
}
```

10 Fórmulas

10.1 Fórmulas de sumatorias

Estas identidades permiten calcular sumas comunes en tiempo constante.

$$\sum_{i=1}^{n} 1 = n$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

Progresión aritmética Para una sucesión que comienza en a y aumenta de d en d:

$$\sum_{i=0}^{n-1}(a+id) = \frac{n}{2}\left(2a+(n-1)d\right).$$

Progresión geométrica

$$\sum_{i=0}^{n-1} r^i = \begin{cases} n, & r = 1, \\ \dfrac{r^n-1}{r-1}, & r \neq 1. \end{cases}$$

Sumas útiles en combinatoria

$$\sum_{k=0}^{n}\binom{n}{k} = 2^n, \qquad \sum_{k=0}^{n} k\binom{n}{k} = n2^{n-1}.$$

10.2 Picks Theorem

Formula

$$A = I + \frac{B}{2} - 1$$

Where:
A=Area of the polygon

I=Number of points inside the polygon
B=Number of points of the border of polygon